`constexpr std::format`

Document #:	P3391R1 [Latest] [Status]
Date:	2025-04-15
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

Since [P3391R0], also noted that using the `L` locale specifier prevents constexpr formatting, and a feature-test macro.

§ 2 Introduction

With the adoption of [P2741R3], `static_assert` can take a `std::string`. And the standard library has a very convenient way of producing a `std::string` by way of `std::format`. Except that it's not `constexpr`, so it's not suitable for that purpose. Let's change that.

`std::format` is specified to use type-erasure, by way of a handle type as specified in 28.5.8.1 [format.arg]/10:

```
namespace std {
  template<class Context>
  class basic_format_arg<Context>::handle {
    const void* ptr_;                                                    //
        exposition only
    void (*format_)(basic_format_parse_context<char_type>&,
```

```
                        Context&, const void*);                          //
          exposition only

       template<class T> explicit handle(T& val) noexcept;               //
          exposition only

      friend class basic_format_arg<Context>;                            //
          exposition only

    public:
        void format(basic_format_parse_context<char_type>&, Context& ctx)
          const;
    };
  }
```

Such a type was unusable during constant evaluate due to the to cast `ptr_` from `void const*` to some `T const*` in the `format_` function. But with the adoption of [P2738R1], that cast is now allowed. And, if such `handles` were constructed in place, such construction is now allowed too with the adoption of [P2747R2].

There's really nothing that stands in the way of making `std::format` fully `constexpr`.

Except… there are two categories of types that we cannot immediately support without further work.

§ 2.1 Floating Point

Integral and floating point types are specified to use `std::to_chars`. However, only one overload of `to_chars` is currently declared `constexpr` in 28.2.1 [charconv.syn]:

```
constexpr to_chars_result to_chars(char* first, char* last,              //
        freestanding
                                   integer-type value, int base = 10);
to_chars_result to_chars(char* first, char* last,                        //
        freestanding
                         bool value, int base = 10) = delete;
to_chars_result to_chars(char* first, char* last,                        //
        freestanding-deleted
                         floating-point-type value);
to_chars_result to_chars(char* first, char* last,                        //
        freestanding-deleted
                         floating-point-type value, chars_format fmt);
to_chars_result to_chars(char* first, char* last,                        //
        freestanding-deleted
                           floating-point-type value, chars_format fmt,
        int precision);
```

That was added by [P2291R3] which explicitly noted the difficulty of implementing floating point support. That paper was adopted in 2021, and a lot has chanced since then (including multiple floating point formatting algorithms). So perhaps this decision should be revisited at some point.

§ 2.2 Chrono Types

The chrono types (all of them) are specified to be formatted as if by streaming through `basic_ostringstream<char>` (see 30.12 [time.format]) and absolutely nothing in that type is `constexpr`. That won't work without further changes — most likely by changing how chrono type formatting works rather than by changing `basic_ostringstream`.

§ 2.3 Other Types

There are a few other types in the standard library that have formatters that rely on functionality that is not currently `constexpr`: `stacktrace_entry`, `filesystem::path`, and `thread::id`. Those will remain non-`constexpr` formattable. Additionally, `void const*` cannot be formatted at compile time because it cannot be converted to an address.

§ 2.4 Implementation Experience

The `{fmt}` library has supported compile-time formatting for a while, just through a different API with the format string annotated: as `format_to(out, FMT_COMPILE("x={}"), x)`. That implementation even supports floating point types at compile time (example), but not the chrono types.

Implementing this in libstdc++ was mostly a matter of marking a lot of functions `constexpr` (which is easy thanks to `-fimplicit-constexpr`). There were only two specific changes I had to make:

1. When assigning into the union, the current implementation does this:

```
template<typename _Tp>
 [[__gnu__::__always_inline__]]
 void
 _M_set(_Tp __v) noexcept
 {
   if constexpr (derived_from<_Tp, _HandleBase>)
     std::construct_at(&_M_handle, __v);
   else
     _S_get<_Tp>(*this) = __v;
 }
```

   In that last assignment, `_S_get<_Tp>(*this)` returns the appropriate member of the `union` for the type `_Tp`. That doesn't work during constant evaluation (although it can probably be made to) since there's no active member yet. So I had to change that to be a new function invoked like `_S_set(*this, __v)`.

2. Like `{fmt}`, libstdc++ has a derived scanner type that is only constructed during constant evaluation time and is only used for parsing. This gets confusing when we also do formatting at compile time, because we don't have that compile-time scanner type, so casting down to it will fail. In libstdc++, this is easy to fix though since there is a member pointer to array of types that is just `nullptr` if that scanner isn't used — so we can check that before downcasting.

And with that, this works (you can see the changes in question on line 3322 for the changed call to `_S_set` and 4392 for the check, at compile-time only, of `_M_types`):

```
template <auto F>
constexpr auto formatted = []{
    static constexpr auto array = []{
        std::array<char, 100> a = {};
        F(a.data());
        return a;
    }();

    return std::string_view(array.data());
}();

static_assert(formatted<[](char* p){std::format_to(p, "x={}", 42);}> ==
        "x=42");
static_assert(formatted<[](char*  p){std::format_to(p,  "x={:{}}",  42,
        5);}> == "x=   42");
```

§ 3 Proposal

Make `std::format` `constexpr`, with the understanding that we will not (yet) be able to format floating point and chrono types during constant evaluation time, nor the locale-aware overloads. The facility is still plenty useful even if we can't format everything quite yet!

§ 3.1 Wording

[Drafting note: I'm introducing the term "constexpr-enabled" to describe the standard library formatters that have a `constexpr format` function. This will include most types, but not the floating-point or chrono types mentioned earlier. As such, there are no changes to 30.12 [time.format] here. Also, most of the wording is simply adding `constexpr` to a lot of places — only the synopses are shown in the diff below.].

Add `constexpr` to a lot of places in 28.5.1 [format.syn]:

```
namespace std {
  // [format.context], class template basic_format_context
  template<class Out, class charT> class basic_format_context;
  using format_context = basic_format_context<unspecified, char>;
  using wformat_context = basic_format_context<unspecified, wchar_t>;

  // [format.args], class template basic_format_args
  template<class Context> class basic_format_args;
  using format_args = basic_format_args<format_context>;
  using wformat_args = basic_format_args<wformat_context>;

  // [format.fmt.string], class template basic_format_string
  template<class charT, class... Args>
    struct basic_format_string;

  template<class charT> struct runtime-format-string {                  //
      exposition only
  private:
```

```
      basic_string_view<charT> str;                                    //
        exposition only
    public:
-     runtime-format-string(basic_string_view<charT> s) noexcept : str(s) {}
+     constexpr runtime-format-string(basic_string_view<charT> s) noexcept :
        str(s) {}
      runtime-format-string(const runtime-format-string&) = delete;
          runtime-format-string& operator=(const runtime-format-string&) =
        delete;
    };
-    runtime-format-string<char> runtime_format(string_view fmt) noexcept {
        return fmt; }
-   runtime-format-string<wchar_t> runtime_format(wstring_view fmt) noexcept
        { return fmt; }
+     constexpr runtime-format-string<char> runtime_format(string_view fmt)
        noexcept { return fmt; }
+      constexpr runtime-format-string<wchar_t> runtime_format(wstring_view
        fmt) noexcept { return fmt; }

    template<class... Args>
                    using   format_string   =   basic_format_string<char,
        type_identity_t<Args>...>;
    template<class... Args>
                  using   wformat_string   =   basic_format_string<wchar_t,
        type_identity_t<Args>...>;

    // [format.functions], formatting functions
    template<class... Args>
-     string format(format_string<Args...> fmt, Args&&... args);
+     constexpr string format(format_string<Args...> fmt, Args&&... args);
    template<class... Args>
-     wstring format(wformat_string<Args...> fmt, Args&&... args);
+     constexpr wstring format(wformat_string<Args...> fmt, Args&&... args);
    template<class... Args>
      string format(const locale& loc, format_string<Args...> fmt, Args&&...
        args);
    template<class... Args>
          wstring  format(const  locale&  loc,  wformat_string<Args...>  fmt,
        Args&&... args);

-   string vformat(string_view fmt, format_args args);
-   wstring vformat(wstring_view fmt, wformat_args args);
+   constexpr string vformat(string_view fmt, format_args args);
+   constexpr wstring vformat(wstring_view fmt, wformat_args args);
    string vformat(const locale& loc, string_view fmt, format_args args);
    wstring vformat(const locale& loc, wstring_view fmt, wformat_args args);

    template<class Out, class... Args>
-     Out format_to(Out out, format_string<Args...> fmt, Args&&... args);
+     constexpr Out format_to(Out out, format_string<Args...> fmt, Args&&...
        args);
    template<class Out, class... Args>
-     Out format_to(Out out, wformat_string<Args...> fmt, Args&&... args);
+          constexpr  Out  format_to(Out  out,  wformat_string<Args...>  fmt,
        Args&&... args);
    template<class Out, class... Args>
```

```
    Out format_to(Out out, const locale& loc, format_string<Args...> fmt,
      Args&&... args);
  template<class Out, class... Args>
    Out format_to(Out out, const locale& loc, wformat_string<Args...> fmt,
      Args&&... args);

  template<class Out>
-   Out vformat_to(Out out, string_view fmt, format_args args);
+   constexpr Out vformat_to(Out out, string_view fmt, format_args args);
  template<class Out>
-   Out vformat_to(Out out, wstring_view fmt, wformat_args args);
+     constexpr  Out  vformat_to(Out  out,  wstring_view  fmt,  wformat_args
      args);
  template<class Out>
       Out  vformat_to(Out  out,  const  locale&  loc,  string_view  fmt,
      format_args args);
  template<class Out>
      Out  vformat_to(Out  out,  const  locale&  loc,  wstring_view  fmt,
      wformat_args args);

  template<class Out> struct format_to_n_result {
    Out out;
    iter_difference_t<Out> size;
  };
  template<class Out, class... Args>
-   format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
+             constexpr   format_to_n_result<Out>   format_to_n(Out   out,
      iter_difference_t<Out> n,
                                                format_string<Args...> fmt,
      Args&&... args);
  template<class Out, class... Args>
-   format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
+             constexpr   format_to_n_result<Out>   format_to_n(Out   out,
      iter_difference_t<Out> n,
                                               wformat_string<Args...> fmt,
      Args&&... args);
  template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                                        const locale& loc,
      format_string<Args...> fmt,
                                         Args&&... args);
  template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                                        const locale& loc,
      wformat_string<Args...> fmt,
                                         Args&&... args);

  template<class... Args>
-   size_t formatted_size(format_string<Args...> fmt, Args&&... args);
+    constexpr size_t formatted_size(format_string<Args...> fmt, Args&&...
      args);
  template<class... Args>
-   size_t formatted_size(wformat_string<Args...> fmt, Args&&... args);
+   constexpr size_t formatted_size(wformat_string<Args...> fmt, Args&&...
      args);
  template<class... Args>
```

```
    size_t formatted_size(const locale& loc, format_string<Args...> fmt,
     Args&&... args);
  template<class... Args>
    size_t formatted_size(const locale& loc, wformat_string<Args...> fmt,
     Args&&... args);

  // [format.formatter], formatter
  template<class T, class charT = char> struct formatter;

  // [format.formatter.locking], formatter locking
  template<class T>
    constexpr bool enable_nonlocking_formatter_optimization = false;

  // [format.formattable], concept formattable
  template<class T, class charT>
    concept formattable = see below;

  template<class R, class charT>
    concept const-formattable-range =                                   //
      exposition only
      ranges::input_range<const R> &&
      formattable<ranges::range_reference_t<const R>, charT>;

  template<class R, class charT>
    using fmt-maybe-const =                                             //
      exposition only
      conditional_t<const-formattable-range<R, charT>, const R, R>;

  // [format.parse.ctx], class template basic_format_parse_context
  template<class charT> class basic_format_parse_context;
  using format_parse_context = basic_format_parse_context<char>;
  using wformat_parse_context = basic_format_parse_context<wchar_t>;

  // [format.range], formatting of ranges
  // [format.range.fmtkind], variable template format_kind
  enum class range_format {
    disabled,
    map,
    set,
    sequence,
    string,
    debug_string
  };

  template<class R>
    constexpr unspecified format_kind = unspecified;

  template<ranges::input_range R>
      requires same_as<R, remove_cvref_t<R>>
    constexpr range_format format_kind<R> = see below;

  // [format.range.formatter], class template range_formatter
  template<class T, class charT = char>
    requires same_as<remove_cvref_t<T>, T> && formattable<T, charT>
  class range_formatter;
```

```
    // [format.range.fmtdef], class template range-default-formatter
    template<range_format K, ranges::input_range R, class charT>
      struct range-default-formatter;                                  //
        exposition only

     // [format.range.fmtmap], [format.range.fmtset], [format.range.fmtstr],
        specializations for maps, sets, and strings
    template<ranges::input_range R, class charT>
      requires (format_kind<R> != range_format::disabled) &&
               formattable<ranges::range_reference_t<R>, charT>
     struct formatter<R, charT> : range-default-formatter<format_kind<R>, R,
        charT> { };

    template<ranges::input_range R>
        requires (format_kind<R> != range_format::disabled)
        inline constexpr bool enable_nonlocking_formatter_optimization<R> =
        false;

    // [format.arguments], arguments
    // [format.arg], class template basic_format_arg
    template<class Context> class basic_format_arg;

    // [format.arg.store], class template format-arg-store
    template<class Context, class... Args> class format-arg-store;        //
        exposition only

    template<class Context = format_context, class... Args>
-     format-arg-store<Context, Args...>
+     constexpr format-arg-store<Context, Args...>
        make_format_args(Args&... fmt_args);
    template<class... Args>
-     format-arg-store<wformat_context, Args...>
+     constexpr format-arg-store<wformat_context, Args...>
        make_wformat_args(Args&... args);

    // [format.error], class format_error
    class format_error;
  }
```

Apply the same changes where these functions are referenced.

Change 28.5.2.2 [format.string.std]/17:

> 17 When the `L` option is used, the form used for the conversion is called the *locale-specific* form. The `L` option is only valid for arithmetic types, and its effect depends upon the type. A call to `format` on a given `formatter` specialization is not a core constant expression if the locale-specific form is specified.

Mark the *`runtime-format-string`* constructor `constexpr` in 28.5.4 [format.fmt.string]:

```
  namespace std {
    template<class charT, class... Args>
```

```
  struct basic_format_string {
  private:
    basic_string_view<charT> str;         // exposition only

  public:
    template<class T> consteval basic_format_string(const T& s);
-     basic_format_string(runtime-format-string<charT> s) noexcept :
        str(s.str) {}
+       constexpr basic_format_string(runtime-format-string<charT> s)
        noexcept : str(s.str) {}

      constexpr basic_string_view<charT> get() const noexcept { return
        str; }
  };
}
```

Add to 28.5.6.4 [format.formatter.spec]:

2 Let `charT` be either `char` or `wchar_t`. Each specialization of `formatter` is either enabled or disabled, as described below. A *debug-enabled* specialization of `formatter` additionally provides a public, constexpr, non-static member function `set_debug_format()` which modifies the state of the `formatter` to be as if the type of the *`std-format-spec`* parsed by the last call to `parse` were `?`. A *constexpr-enabled* specialization of `formatter` has its `format` member function declared `constexpr`. Each header that declares the template `formatter` provides the following enabled specializations:

(2.1) — The debug-enabled and constexpr-enabled specializations

```
template<> struct formatter<char, char>;
template<> struct formatter<char, wchar_t>;
template<> struct formatter<wchar_t, wchar_t>;
```

(2.2) — For each `charT`, the debug-enabled and constexpr-enabled string type specializations

```
template<> struct formatter<charT*, charT>;
template<> struct formatter<const charT*, charT>;
template<size_t N> struct formatter<charT[N], charT>;
template<class traits, class Allocator>
  struct formatter<basic_string<charT, traits, Allocator>, charT>;
template<class traits>
  struct formatter<basic_string_view<charT, traits>, charT>;
```

(2.3) — For each `charT`, for each cv-unqualified arithmetic type `ArithmeticT` other than `char`, `wchar_t`, `char8_t`, `char16_t`, or `char32_t`, a specialization that is constexpr-enabled unless `ArithmeticT` is a floating-point type

```
template<> struct formatter<ArithmeticT, charT>;
```

(2.4) — For each `charT`, the constexpr-enabled pointer type specializations

```
template<> struct formatter<nullptr_t, charT>;
```

(2.5) — For each `charT`, the pointer type specializations

```
template<> struct formatter<void*, charT>;
template<> struct formatter<const void*, charT>;
```

The parse member functions of these formatters interpret the format specification as a *std-format-spec* as described in [format.string.std].

Mark `basic_format_context` as mostly `constexpr` in 28.5.6.7 [format.context] (everything but `locale()`, and repeated in the specification of these functions):

```
namespace std {
  template<class Out, class charT>
  class basic_format_context {
     basic_format_args<basic_format_context> args_;       // exposition
       only
     Out out_;                                            // exposition
       only

     basic_format_context(const basic_format_context&) = delete;
       basic_format_context& operator=(const basic_format_context&) =
       delete;

   public:
     using iterator = Out;
     using char_type = charT;
     template<class T> using formatter_type = formatter<T, charT>;

-        basic_format_arg<basic_format_context> arg(size_t id) const
       noexcept;
+      constexpr basic_format_arg<basic_format_context> arg(size_t id)
       const noexcept;
     std::locale locale();

-    iterator out();
-    void advance_to(iterator it);
+    constexpr iterator out();
+    constexpr void advance_to(iterator it);
   };
 }
```

Mark `range_formatter::format` as `constexpr` in 28.5.7.2 [format.range.formatter] and repeated in its specification:

```
namespace std {
  template<class T, class charT = char>
    requires same_as<remove_cvref_t<T>, T> && formattable<T, charT>
  class range_formatter {
                            formatter<T,     charT>     underlying_;
     // exposition only
    basic_string_view<charT> separator_ = STATICALLY-WIDEN<charT>(",
     ");      // exposition only
```

```
            basic_string_view<charT>  opening-bracket_  =  STATICALLY-
        WIDEN<charT>("["); // exposition only
            basic_string_view<charT>  closing-bracket_  =  STATICALLY-
        WIDEN<charT>("]"); // exposition only

    public:
        constexpr  void  set_separator(basic_string_view<charT>  sep)
        noexcept;
      constexpr void set_brackets(basic_string_view<charT> opening,
                                  basic_string_view<charT> closing)
        noexcept;
        constexpr  formatter<T,  charT>& underlying()  noexcept  {  return
        underlying_; }
      constexpr const formatter<T, charT>& underlying() const noexcept {
        return underlying_; }

      template<class ParseContext>
        constexpr typename ParseContext::iterator
          parse(ParseContext& ctx);

      template<ranges::input_range R, class FormatContext>
          requires formattable<ranges::range_reference_t<R>, charT> &&
                   same_as<remove_cvref_t<ranges::range_reference_t<R>>,
        T>
-       typename FormatContext::iterator
+       constexpr typename FormatContext::iterator
          format(R&& r, FormatContext& ctx) const;
    };
  }
```

And likewise with *`range-default-formatter`* in 28.5.7.3 [format.range.fmtdef]:

```
  namespace std {
    template<ranges::input_range R, class charT>
     struct range-default-formatter<range_format::sequence, R, charT> {
        // exposition only
    private:
                using   maybe-const-r   =   fmt-maybe-const<R,   charT>;
        // exposition only
         range_formatter<remove_cvref_t<ranges::range_reference_t<maybe-
        const-r>>,
                                                  charT>  underlying_;
        // exposition only

    public:
        constexpr  void  set_separator(basic_string_view<charT>  sep)
        noexcept;
      constexpr void set_brackets(basic_string_view<charT> opening,
                                  basic_string_view<charT> closing)
        noexcept;

      template<class ParseContext>
        constexpr typename ParseContext::iterator
          parse(ParseContext& ctx);

      template<class FormatContext>
```

```
-       typename FormatContext::iterator
+       constexpr typename FormatContext::iterator
          format(maybe-const-r& elems, FormatContext& ctx) const;
    };
  }
```

And the *range-default-formatter* for maps in 28.5.7.4 [format.range.fmtmap]:

```
  namespace std {
    template<ranges::input_range R, class charT>
    struct range-default-formatter<range_format::map, R, charT> {
    private:
                using   maybe-const-map   =   fmt-maybe-const<R,   charT>;
        // exposition only
                                          using       element-type       =
        // exposition only
        remove_cvref_t<ranges::range_reference_t<maybe-const-map>>;
                  range_formatter<element-type,   charT>   underlying_;
        // exposition only

    public:
      constexpr range-default-formatter();

      template<class ParseContext>
        constexpr typename ParseContext::iterator
          parse(ParseContext& ctx);

      template<class FormatContext>
-       typename FormatContext::iterator
+       constexpr typename FormatContext::iterator
          format(maybe-const-map& r, FormatContext& ctx) const;
    };
  }
```

And the *range-default-formatter* for sets in 28.5.7.5 [format.range.fmtset]:

```
  namespace std {
    template<ranges::input_range R, class charT>
    struct range-default-formatter<range_format::set, R, charT> {
    private:
                using   maybe-const-set   =   fmt-maybe-const<R,   charT>;
        // exposition only
         range_formatter<remove_cvref_t<ranges::range_reference_t<maybe-
        const-set>>,
                                                  charT>   underlying_;
        // exposition only

    public:
      constexpr range-default-formatter();

      template<class ParseContext>
        constexpr typename ParseContext::iterator
          parse(ParseContext& ctx);
```

```
      template<class FormatContext>
-       typename FormatContext::iterator
+       constexpr typename FormatContext::iterator
          format(maybe-const-set& r, FormatContext& ctx) const;
    };
  }
```

And the *`range-default-formatter`* for strings in 28.5.7.6 [format.range.fmtstr]:

```
  namespace std {
    template<range_format K, ranges::input_range R, class charT>
                requires   (K   ==   range_format::string   ||   K   ==
        range_format::debug_string)
    struct range-default-formatter<K, R, charT> {
    private:
                formatter<basic_string<charT>,   charT>   underlying_;
        // exposition only

    public:
      template<class ParseContext>
        constexpr typename ParseContext::iterator
          parse(ParseContext& ctx);

      template<class FormatContext>
-       typename FormatContext::iterator
+       constexpr typename FormatContext::iterator
          format(see below& str, FormatContext& ctx) const;
    };
  }
```

The `basic_format_arg` class template can be made entirely `constexpr` too, in 28.5.8.1 [format.arg]:

```
  namespace std {
    template<class Context>
    class basic_format_arg {
    public:
      class handle;

    private:
                using   char_type   =   typename   Context::char_type;
        // exposition only

      variant<monostate, bool, char_type,
              int, unsigned int, long long int, unsigned long long int,
              float, double, long double,
              const char_type*, basic_string_view<char_type>,
                                    const   void*,   handle>   value;
        // exposition only

-       template<class  T>  explicit  basic_format_arg(T&  v)  noexcept;
        // exposition only
```

```
+       template<class T> constexpr explicit basic_format_arg(T& v)
        noexcept;       // exposition only

    public:
-     basic_format_arg() noexcept;
+     constexpr basic_format_arg() noexcept;

-     explicit operator bool() const noexcept;
+     constexpr explicit operator bool() const noexcept;

      template<class Visitor>
-       decltype(auto) visit(this basic_format_arg arg, Visitor&& vis);
+           constexpr decltype(auto) visit(this basic_format_arg arg,
        Visitor&& vis);
      template<class R, class Visitor>
-       R visit(this basic_format_arg arg, Visitor&& vis);
+       constexpr R visit(this basic_format_arg arg, Visitor&& vis);
    };
  }
```

And the `handle` type introduced in 28.5.8.1 [format.arg]/10:

> 10 The class handle allows formatting an object of a user-defined type.

```
  namespace std {
    template<class Context>
    class basic_format_arg<Context>::handle {
       const void* ptr_;                                          //
         exposition only
       void (*format_)(basic_format_parse_context<char_type>&,
                       Context&, const void*);                    //
         exposition only
-      template<class T> explicit handle(T& val) noexcept;         //
         exposition only
+      template<class T> constexpr explicit handle(T& val) noexcept; //
         exposition only
       friend class basic_format_arg<Context>;                    //
         exposition only
    public:
-     void format(basic_format_parse_context<char_type>&, Context& ctx)
        const;
+       constexpr void format(basic_format_parse_context<char_type>&,
        Context& ctx) const;
    };
  }
```

And `basic_format_args` in 28.5.8.3 [format.args]:

```
  namespace std {
    template<class Context>
    class basic_format_args {
      size_t size_;                              // exposition only
      const basic_format_arg<Context>* data_;    // exposition only
```

```
    public:
      template<class... Args>
-           basic_format_args(const format-arg-store<Context, Args...>&
        store) noexcept;
+           constexpr basic_format_args(const format-arg-store<Context,
        Args...>& store) noexcept;

-     basic_format_arg<Context> get(size_t i) const noexcept;
+     constexpr basic_format_arg<Context> get(size_t i) const noexcept;
    };

    template<class Context, class... Args>
             basic_format_args(format-arg-store<Context, Args...>) ->
        basic_format_args<Context>;
  }
```

And the tuple formatter in 28.5.9 [format.tuple]:

```
  namespace std {
    template<class charT, formattable<charT>... Ts>
    struct formatter<pair-or-tuple<Ts...>, charT> {
    private:
           tuple<formatter<remove_cvref_t<Ts>, charT>...> underlying_;
        // exposition only
       basic_string_view<charT> separator_ = STATICALLY-WIDEN<charT>(",
        ");      // exposition only
            basic_string_view<charT> opening-bracket_ = STATICALLY-
        WIDEN<charT>("("); // exposition only
            basic_string_view<charT> closing-bracket_ = STATICALLY-
        WIDEN<charT>(")"); // exposition only

    public:
           constexpr void set_separator(basic_string_view<charT> sep)
        noexcept;
      constexpr void set_brackets(basic_string_view<charT> opening,
                                  basic_string_view<charT> closing)
        noexcept;

      template<class ParseContext>
        constexpr typename ParseContext::iterator
          parse(ParseContext& ctx);

      template<class FormatContext>
-       typename FormatContext::iterator
+       constexpr typename FormatContext::iterator
          format(see below& elems, FormatContext& ctx) const;
    };

    template<class... Ts>
                                       inline      constexpr      bool
        enable_nonlocking_formatter_optimization<pair-or-tuple<Ts...>>
        =
        (enable_nonlocking_formatter_optimization<Ts> && ...);
  }
```

Lastly, `format_error` for now will remain untouched — pending a combination of [P3068R1] and [P3295R0], since `runtime_error` will also have to be marked.

Mark the `vector<bool>::reference` formatter `constexpr` in 23.3.14.2 [vector.bool.fmt]:

```
  namespace std {
    template<class T, class charT>
      requires is-vector-bool-reference<T>
    struct formatter<T, charT> {
    private:
      formatter<bool, charT> underlying_;       // exposition only

    public:
      template<class ParseContext>
        constexpr typename ParseContext::iterator
          parse(ParseContext& ctx);

      template<class FormatContext>
-       typename FormatContext::iterator
+       constexpr typename FormatContext::iterator
          format(const T& ref, FormatContext& ctx) const;
    };
  }
```

And the container adaptors in 23.6.13 [container.adaptors.format]:

1 For each of `queue`, `priority_queue`, and `stack`, the library provides the following constexpr-enabled `formatter` specialization where *`adaptor-type`* is the name of the template:

```
  namespace std {
       template<class charT, class T, formattable<charT> Container,
        class... U>
    struct formatter<adaptor-type<T, Container, U...>, charT> {
    private:
                                  using     maybe-const-container     =
        // exposition only
        fmt-maybe-const<Container, charT>;
                                   using     maybe-const-adaptor     =
        // exposition only
                         maybe-const<is_const_v<maybe-const-container>,
        // see [ranges.syn]
                    adaptor-type<T, Container, U...>>;
             formatter<ranges::ref_view<maybe-const-container>,  charT>
        underlying_;    // exposition only

    public:
      template<class ParseContext>
        constexpr typename ParseContext::iterator
          parse(ParseContext& ctx);

      template<class FormatContext>
```

```
-       typename FormatContext::iterator
+       constexpr typename FormatContext::iterator
          format(maybe-const-adaptor& r, FormatContext& ctx) const;
    };
  }
```

3.2 Feature-Test Macro

Add a new `__cpp_lib_constexpr_format` to 17.3.2 [version.syn]:

```
#define __cpp_lib_constexpr_format 2025XXL // also in <format>
```

4 References

[P2291R3] Daniil Goncharov, Karaev Alexander. 2021-09-23. Add Constexpr Modifiers to Functions to_chars and from_chars for Integral Types in Header.
https://wg21.link/p2291r3

[P2738R1] Corentin Jabot, David Ledger. 2023-02-13. constexpr cast from void*: towards constexpr type-erasure.
https://wg21.link/p2738r1

[P2741R3] Corentin Jabot. 2023-06-16. user-generated static_assert messages.
https://wg21.link/p2741r3

[P2747R2] Barry Revzin. 2024-03-19. constexpr placement new.
https://wg21.link/p2747r2

[P3068R1] Hana Dusíková. 2024-03-30. Allowing exception throwing in constant-evaluation.
https://wg21.link/p3068r1

[P3295R0] Ben Craig. 2024-05-21. Freestanding constexpr containers and constexpr exception types.
https://wg21.link/p3295r0

[P3391R0] Barry Revzin. 2024-09-12. constexpr std::format.
https://wg21.link/p3391r0